A Study of Lightweight Post-Deployment Adaptation for Sequential Recommendation

MSE 720 Project Presentation

Nayeema Nonta

Outline

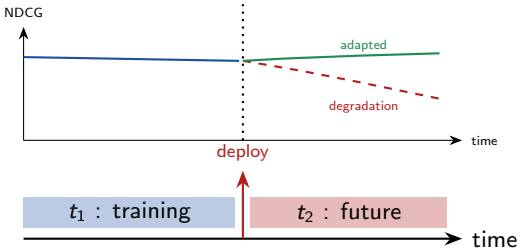
1. Motivation & Problem
2. Model & Approach
3. Experimental Setup
4. Experimental Setup
5. Results
6. Discussion & Conclusions

Model Drift Post Deployment

Setup: a sequential recommender is trained on an earlier period, then used on later user behaviour.

After deployment, the data can shift:

- user interests evolve
- local item-to-item transitions change
- the short-range sequence context changes



Problem: A model trained once on t_1 can lose ranking accuracy on t_2.

Naive solution: retrain or continually maintain the full model.

Issue: more data, more compute, more engineering.

Research Question & Compared Methods

Research Question

When a sequential recommender degrades after deployment, which **lightweight adaptation mechanisms** actually recover future ranking performance?

Three methods compared:

1. **Last-Block Fine-Tuning**

   (**Weight-space**: traditional parameter updates)

2. **Prototype Steering**

   (**Activation-space**: shifting representations, frozen backbone)

3. **Context-Gated Adapter**

   (**Activation-space**: conditional gating, frozen backbone)

Motivation for Methods

Recent RecSys work uses post-hoc intervention for:

- activation steering for popularity bias control (Ahmadov and Mansoury, 2026)
- inference-time steering for user-specific popularity alignment (Schirmer et al., 2026)
- SAE-based controllable recommendation (Klenitskiy et al., 2025; Spišák et al., 2026)

What these works solve

Alignment, **bias mitigation**, and **controllable recommendation** through post-hoc intervention on recommender representations

Key distinction

Alignment = match a chosen preference notion
Adaptation = recover performance under changed data

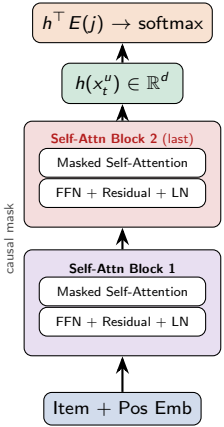
This study applies steering concept to recover **future ranking performance** when the underlying distribution shifts.

SASRec Backbone

Self-Attentive Sequential Recommender

(Kang & McAuley, 2018)

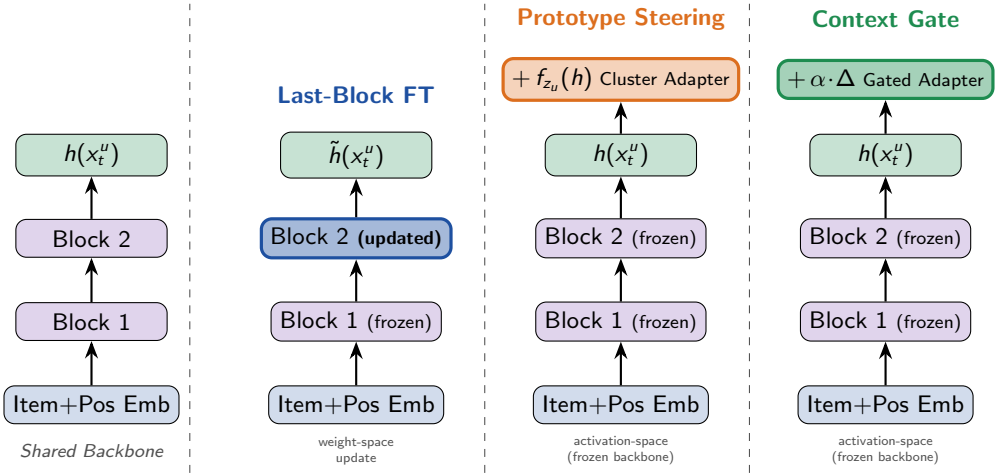
- **Input:** Prefix $x_t^u = (i_1^u, \ldots, i_{t-1}^u)$, $n = 128$
- **Architecture:** 2 causal self-attn blocks
- **Specs:** $d = 64$, 2 heads
- **Scoring:** $s(j \mid x_t^u) = h(x_t^u)^\top E(j)$
- **Loss:** Full-vocabulary cross-entropy



Key Property

Adaptive weighting of past items via **self-attention** mechanisms.

Adaptation Methods Overview



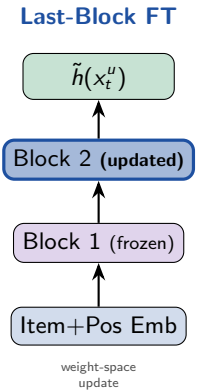
Method 1 — Last-Block Fine-Tuning

Idea: update only the final SASRec block.

The first block remains fixed and forms context-aware representations from item and positional embeddings. The second block is updated to adapt higher-level sequential dynamics.

Produced hidden state

$\tilde{h}(x_t^u)$ = output of the partially updated backbone



Method 2 — Prototype Steering

Idea: keep the backbone frozen and apply a cluster-specific adapter.

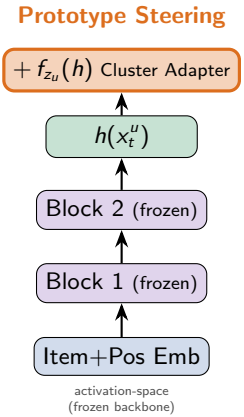
$$\tilde{h}(x_t^u) = h(x_t^u) + f_{z_u}(h(x_t^u))$$

$$f_z(h) = W_{2,z}\,\rho(W_{1,z}\,h + b_{1,z}) + b_{2,z}$$

Users are clustered offline using standardized drift features:

- item-profile cosine distance
- hidden-state cosine distance
- top-k item Jaccard drift

K clusters $\Rightarrow$ K small adapters.



Method 3 — Context-Gated Adapter

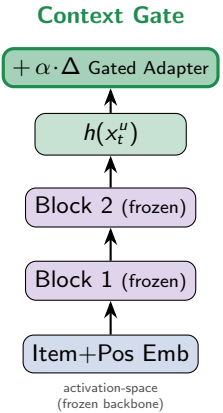
Idea: keep the backbone frozen and apply a shared residual correction with a scaling gate.

$$\tilde{h}(x_t^u) = h(x_t^u) + \alpha(x_t^u)\,\Delta(x_t^u)$$

$$\Delta(x_t^u) = W_2\,\rho(W_1 h(x_t^u) + b_1) + b_2$$

$$\alpha(x_t^u) = \sigma(w^\top h(x_t^u) + b)$$

$\Delta(x_t^u)$ gives the correction direction, and $\alpha(x_t^u) \in (0,1)$ controls how strongly it is applied.



Shared Training Objective

All adaptation methods use the same full-vocabulary cross-entropy loss.

For prefix x_t^u and target item i_t^u:

$$\mathcal{L}_{\text{adapt_CE}}(x_t^u, i_t^u) = -\log \frac{\exp\left(\tilde{h}(x_t^u)^\top E(i_t^u)\right)}{\sum_{j \in \mathcal{I}} \exp\left(\tilde{h}(x_t^u)^\top E(j)\right)}$$

Why use CE over the full vocabulary?

It provides stronger gradient signals than sampled-negative objectives, because every item ranked above the target contributes to the update.

Experimental Setup – Dataset

Dataset: 30Music (Turrin et al., 2015)

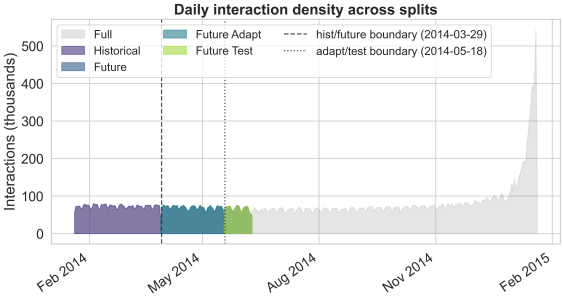
- 29.4M listening events
- 45.1K users
- 3.67M unique observed tracks
- interaction window: 2014-01-20 to 2015-01-20

Why this dataset?

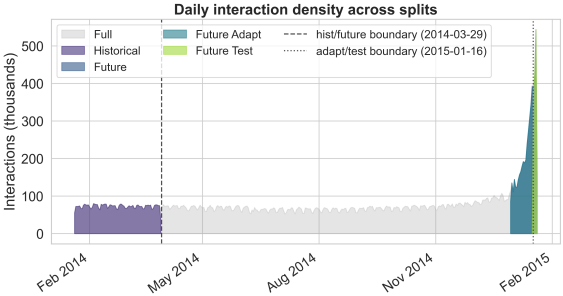
- large-scale music listening dataset
- long user histories
- strong sequential structure for next-item prediction (Klenitskiy et al.,2024)

Experimental Setup — Temporal Split Regimes

Contiguous split



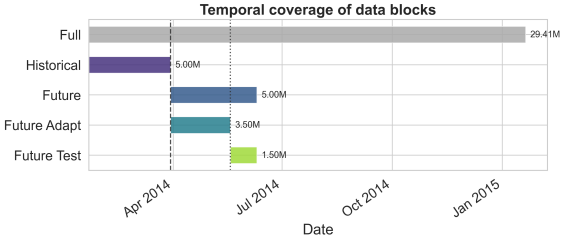
Tail split



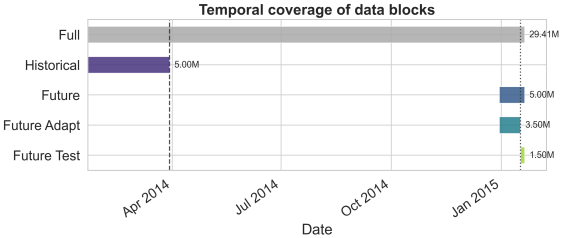
Two temporal adaptation regimes:
contiguous = shorter temporal gap, tail = stronger shift

Experimental Setup — Temporal Split Regimes

Contiguous split



Tail split



Experimental Setup — Isolating Preference Drift

Challenge: many items appearing in t_2 are not in the t_1 training catalog.

Problem

Out-of-vocabulary items can dominate the performance drop, making it hard to separate **catalog expansion** from **user-behaviour drift**.

Overlap-item regime

After iterative 5-core filtering on t_1, restrict t_2 to items that remain in the post-k-core t_1 catalog.

5-core filtering:

- repeatedly removes users with fewer than 5 interactions and items with fewer than 5 occurrences until the dataset stabilizes.

Then focus on high-drift users:

- score users by historical-to-future drift, retain the top 10%

Drift score combines:

- item-frequency profile change, hidden-state representation change, top-k item change

Sanity Check: Does Filtering Preserve Sequential Structure?

Concern: overlap-item filtering and high-drift filtering may weaken the sequential signal that motivates using SASRec.

Test: use the sequential-rules procedure of Klenitskiy et al., 2024.

How it works

- Count frequent **2-gram** rules: $i_{t-1} \rightarrow i_t$
- Count frequent **3-gram** rules: $(i_{t-2}, i_{t-1}) \rightarrow i_t$
- Randomly **shuffle each user's context** but keep the holdout item fixed
- Count the rules again after shuffling

Interpretation

If rule counts collapse after shuffling, the subset has genuine **order-dependent signal**.
Example: -95% means about 95% of the sequential rules disappear after the order is destroyed.

Sanity Check Results

Question: after overlap-item and high-drift filtering, is the remaining subset still sequential enough for SASRec?

Split	Subset	$\Delta^{(2)}$	$\Delta^{(3)}$	Verdict
Contiguous	Raw future test	-92.9%	-95.9%	**Strong**
Contiguous	Overlap + k-core	-92.5%	-95.1%	**Strong**
Contiguous	High-drift k-core	-76.8%	-92.3%	**Moderate**
Tail	Raw future test	-93.9%	-95.9%	**Strong**
Tail	Overlap + k-core	-94.0%	-96.3%	**Strong**
Tail	High-drift k-core	-62.3%	-66.9%	**Weak**

Takeaway

Overlap-item filtering does not destroy sequential structure. The main weakening comes from the **high-drift user subset**, especially in the **tail** split.

Experimental Setup – Evaluation Protocol & Metrics

Evaluation protocol:

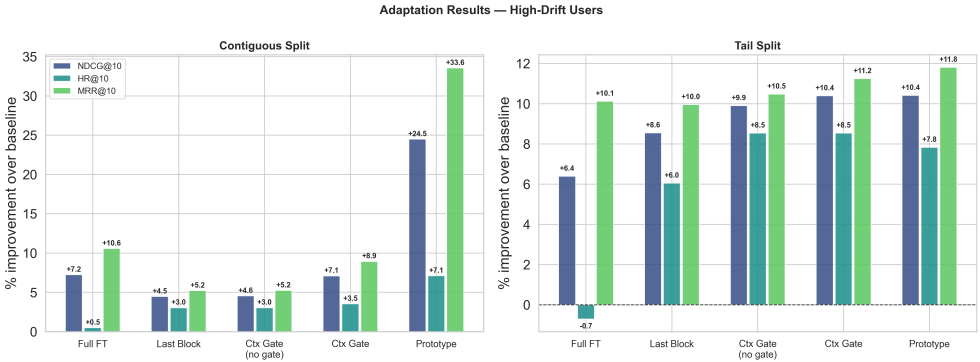
- Ground-truth ranked against **100 sampled negatives**
- Same candidate sets for all methods (seed 42) — paired comparison
- 10 independent seeds for statistical significance

Metrics:

- NDCG — rewards placing the target item higher in the ranking
- HR — did the target appear in the top 10?
- MRR — reciprocal of the target's rank

Main Results — High-Drift Users

All three lightweight adapters improve over the frozen SASRec backbone on both splits.



- **Contiguous:** Prototype Steering gives the strongest recovery.
- **Tail:** method gaps narrow; Context Gate and Last-Block FT move closer.

Main Results — High-Drift Users

Table: Adaptation results on high-drift users. **Bold** = best adapted value per split per metric. † = frozen backbone (adapter only). Baseline is the frozen SASRec backbone evaluated on the same candidate sets.

	Contiguous Split									**Tail Split**								
	NDCG@10			HR@10			MRR@10			NDCG@10			HR@10			MRR@10		
Method	Base	Adpt	%Δ	Base	Adpt	%Δ	Base	Adpt	%Δ	Base	Adpt	%Δ	Base	Adpt	%Δ	Base	Adpt	%Δ
Full FT	0.5874	0.6299	**+7.25%**	0.8419	0.8462	**+0.51%**	0.5072	0.5609	**+10.57%**	0.4676	0.4975	**+6.39%**	0.7060	0.7010	-0.71%	0.3934	0.4333	**+10.12%**
Last Block FT	0.5874	0.6137	**+4.48%**	0.8419	0.8675	**+3.05%**	0.5072	0.5337	**+5.22%**	0.4676	0.5076	**+8.55%**	0.7060	0.7487	**+6.05%**	0.3934	0.4326	**+9.96%**
Context Gate (no gate)†	0.5874	0.6142	**+4.56%**	0.8419	0.8675	**+3.05%**	0.5072	0.5338	**+5.24%**	0.4676	0.5139	**+9.90%**	0.7060	**0.7663**	**+8.54%**	0.3934	0.4347	**+10.48%**
Context Gate†	0.5874	0.6291	**+7.10%**	0.8419	0.8718	**+3.55%**	0.5072	0.5526	**+8.94%**	0.4676	0.5162	**+10.39%**	0.7060	**0.7663**	**+8.54%**	0.3934	0.4377	**+11.25%**
Prototype Steering†	0.5874	**0.7313**	**+24.49%**	0.8419	**0.9017**	**+7.11%**	0.5072	**0.6775**	**+33.56%**	0.4676	**0.5163**	**+10.41%**	0.7060	0.7613	**+7.83%**	0.3934	**0.4399**	**+11.81%**

Main Results — High-Drift Users, Why Does Full FT Underperform?

Full FT is the **most expressive** method, yet it is beaten by all lightweight adapters on the tail split.

Two compounding reasons:

1. **Overfitting on a tiny, biased training set.** The adaptation data is the top-10% high-drift users of the 70% t_2 adapt split. Fine-tuning 12.4M parameters on this memorises drift-specific sequences rather than learning a general correction.
2. **Corrupting general sequential capacity.** Full FT overwrites attention weights and item embeddings that encode *universal* transition patterns. Even drifted t_2 users still depend on these patterns; corrupting them degrades t_2 test performance. Lightweight adapters avoid this by leaving the backbone **fully frozen**.

Key takeaway

With limited adaptation data, **parameter-efficiency is a regularizer**, not just a cost saving.

Results — Multi-Seed Stability

The gains are not an artifact of one seed.

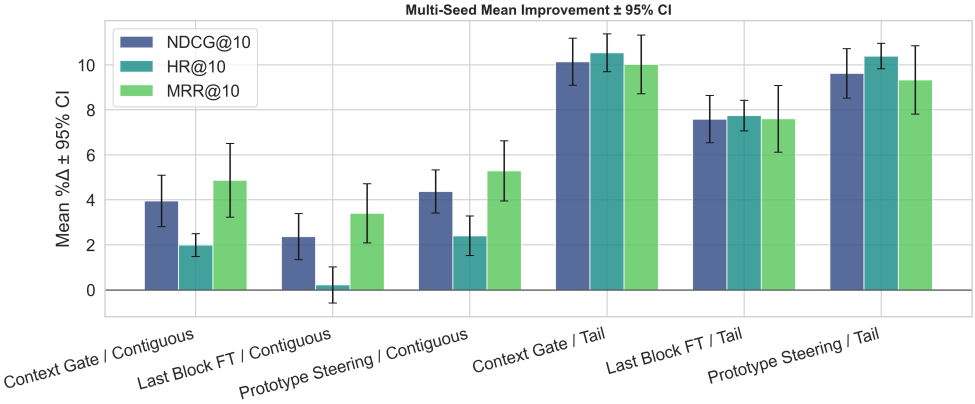


Figure: Mean NDCG@10 improvement $\pm$ 95% confidence interval across 10 random seeds for each method and split.

Results — Multi-Seed Stability

The gains are not an artifact of one seed.

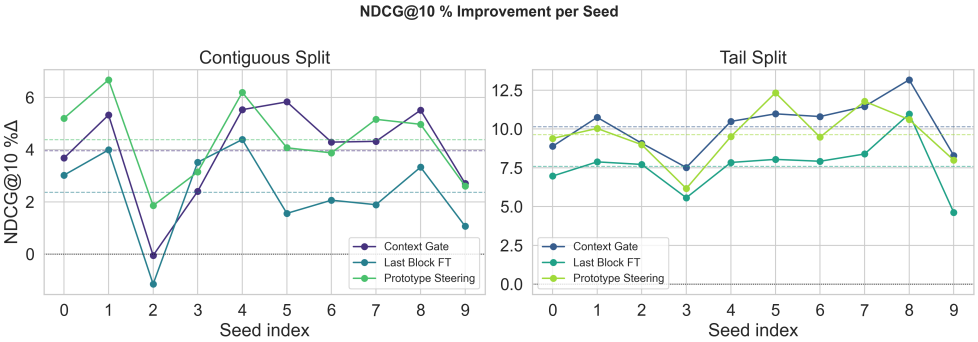


Figure: Multi-seed run line-chart.

Results – Hyperparameter Sensitivity: Cluster Count K

Prototype Steering shows a split-dependent hyperparameter effect.

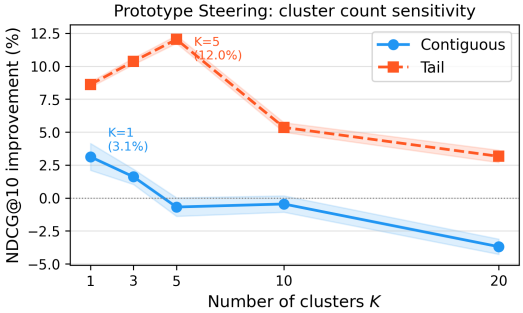


Figure: NDCG@10 improvement (%) of Prototype Steering as a function of cluster count K, averaged over all other hyperparameter settings.

- **Contiguous:** Optimal at $K = 1$. Suggests drifted users behave **cohesively**.
- **Tail:** Optimal at $K \approx 5$. Suggests drift is more **heterogeneous**.
- **Overfitting:** High K hurts as each cluster lacks sufficient data.

Results: Best Hyperparameter Configurations (High Drift Users)

Method	**Split**	**Best Config**	**NDCG@10 Δ**
Last Block FT	Contiguous	lr=5e-4, ep=50, LN=True, wd=0.0	+4.48%
	Tail	lr=1e-3, ep=50, LN=True, wd=1e-4	+8.55%
Context Gate	Contiguous	bd=8, lr=5e-4, ep=10	+7.10%
	Tail	bd=32, lr=1e-3, ep=20	+10.39%
Proto Steering	Contiguous	K=1, bd=16, lr=5e-4, ep=20	+4.58%
	Tail	K=5, bd=64, lr=1e-3, ep=20	+12.56%

- Higher LR optimal on tail split (1e-3 vs 5e-4) — more aggressive updates needed for larger distributional shift
- Context Gate converges quickly (10 epochs contiguous); Last Block FT benefits from longer training (50 epochs)

Results — Generalization Beyond High-Drift Users

Three conditions:

- **HD→HD**: train & eval on high-drift subset
- **HD→ALL**: train on HD, eval all users
- **ALL→ALL**: train & eval on full population

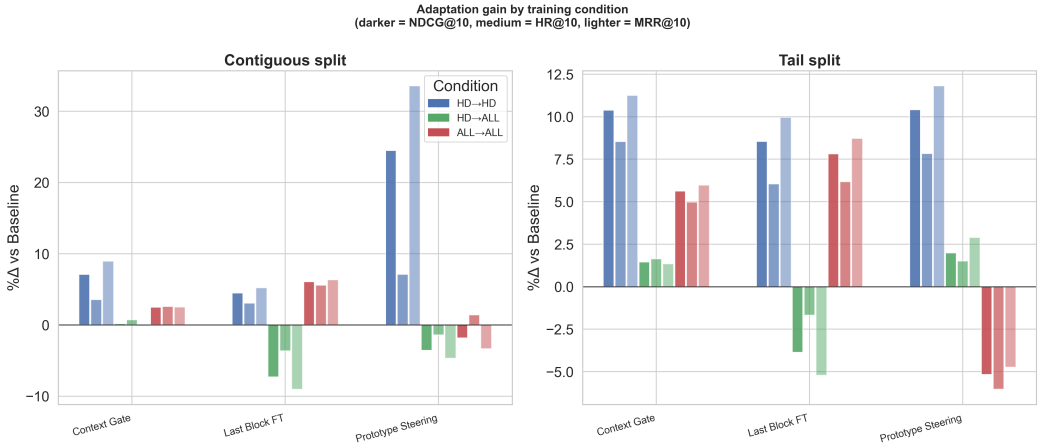
Context Gate: HD→ALL ≈ 0 — safe for the full population (does not hurt); ALL→ALL scales well.

Last Block FT: HD→ALL -7.3% (needs more representative data); **strongest** ALL→ALL ($+7.8\%$ tail).

Prototype Steering: ALL→ALL *hurts* (-5.2% tail) — used same hyperparams as HD which could explain low performance

Results — Generalization Beyond High-Drift Users

We test whether adaptation targeted at drifted users also transfers to the broader population.



Results — Computational Efficiency

Method	Params	Epochs	GPU MB	Time (s)	NDCG@10%Δ (C)	NDCG@10%Δ (T)
Full FT	12,373,952	20	1381	17.8	**+7.25%**	**+6.39%**
Last Block FT	25,344	50	1240	36.7	**+4.48%**	**+8.55%**
Context Gate†	1,161	10	1239	6.6	**+7.10%**	**+10.39%**
Prototype Steering†	2,128	20	1240	14.7	**+24.49%**	**+10.41%**

- Activation-space methods use only **1–2K trainable parameters**, versus **25K** for Last-Block FT and **12.4M** for Full FT.
- **Context Gate is the fastest** at 6.6 s
- Context Gate converges in just 10 epochs vs. 50 for Last-Block FT, driving the total wall-time advantage.
- Despite this, both activation-space methods match or exceed the heavier baselines in ranking gain.

Takeaway

The best deployment trade-off comes from activation-space adaptation: **much smaller, faster, and still highly effective**.

Limitations

Scope

- **Single dataset** (30Music). Results may differ on datasets with weaker sequential structure or sparser interaction histories.
- **Single backbone** (SASRec, 2-block, $d = 64$). Larger or differently-structured backbones may respond differently to activation-space corrections.

Evaluation

- **Single seed sampled negatives** (100 items). Only evaluated on seed=42, so same 100 samples are always used.
- **Overlapping/High-drift subset** (top 10%) is a design choice; not reflective of a real-life setup since most of the drift is from new-items.

Future directions

Full-catalog evaluation · multi-dataset benchmarking · larger backbone architectures

Conclusions

1. **Activation-space adapters match or beat Full FT** while using 4–5 orders of magnitude fewer parameters (1K–2K vs 12M).
2. **Frozen-backbone methods provide safety** that weight-space methods do not — Context Gate causes $\approx 0\%$ NDCG change on stable users; Last Block FT causes -7.3% when restricted to the HD subset.

Appendix: Full Notation Table

Table: Notation.

Symbol	Meaning
$\mathcal{I}$	item catalog
S^u	interaction sequence of user u
x_t^u	prefix used to predict the next item
i_t^u	observed next item after prefix x_t^u
n	maximum sequence length
d	backbone hidden dimension
r	adapter bottleneck dimension
K	number of prototype clusters
$h(x_t^u) \in \mathbb{R}^d$	backbone hidden state for prefix x_t^u
$\tilde{h}(x_t^u) \in \mathbb{R}^d$	adapted hidden state
$E(j) \in \mathbb{R}^d$	embedding of item j
z_u	cluster assignment for user u
$\Delta(x_t^u) \in \mathbb{R}^d$	residual correction vector
$\alpha(x_t^u) \in (0, 1)$	scalar gate for the correction magnitude
$W_1 \in \mathbb{R}^{r \times d}$, $W_2 \in \mathbb{R}^{d \times r}$	bottleneck adapter weights
$\rho(\cdot)$	GELU activation

Appendix: Hyperparameter Search Spaces

Method	HP	Values	Configs
Last Block FT	lr epochs last layernorm weight decay	1e-3, 5e-4, 1e-4, 5e-5 20, 50 False, True 0.0, 1e-4	32
Context Gate	bottleneck dim lr epochs	4, 8, 16, 32 1e-3, 5e-4, 1e-4 10, 20	24
Prototype Steering	num clusters K bottleneck dim lr epochs	1, 3, 5, 10, 20 16, 32, 64 1e-3, 5e-4 20, 50	60

Appendix: Loss Objectives

Backbone pretraining loss (on t_1):

$$\mathcal{L}_{\mathrm{CE}}(x_t^u, i_t^u) = -\log \frac{\exp\big(h(x_t^u)^\top E(i_t^u)\big)}{\sum_{j\in\mathcal{I}} \exp(h(x_t^u)^\top E(j))}$$

Adaptation loss (on t_2 adapt split, same form for all methods):

$$\mathcal{L}_{\mathrm{CE}}^{\mathrm{adapt}}(x_t^u, i_t^u) = -\log \frac{\exp\Big(\tilde{h}(x_t^u)^\top E(i_t^u)\Big)}{\sum_{j\in\mathcal{I}} \exp\Big(\tilde{h}(x_t^u)^\top E(j)\Big)}$$

Both use **full-vocabulary cross-entropy** for stronger gradient signal over the complete catalog (vs. sampled negatives used in the original SASRec).

Appendix: Drift Scoring Details

Three drift signals:

Item-profile drift:

$$d_u^{\text{item}} = 1 - \frac{\langle c_u^{(1)}, c_u^{(2)} \rangle}{\|c_u^{(1)}\|_2 \|c_u^{(2)}\|_2}$$

Hidden-state drift:

$$d_u^{\text{hid}} = 1 - \frac{\langle h_u^{(1)}, h_u^{(2)} \rangle}{\|h_u^{(1)}\|_2 \|h_u^{(2)}\|_2}$$

Top-k item drift (Jaccard):

$$d_u^{\text{topk}} = 1 - \frac{|A_u^{(1)} \cap A_u^{(2)}|}{|A_u^{(1)} \cup A_u^{(2)}|}$$

Combined drift score:

$$z_{u,j} = \frac{d_{u,j} - \mu_j}{\sigma_j}, \quad d_u^{\text{comb}} = \frac{1}{3} \sum_j z_{u,j}$$

Standardize each signal independently, then average.

Top **10%** of users by d^{comb} form the high-drift subset.

Purpose: ensures adaptation is assessed on users who actually change, not on stable users who mask the signal.

Appendix: User Clustering for Prototype Steering

Feature vector per user:

$$x_u = \left[d_u^{\text{item}},\ d_u^{\text{hid}},\ d_u^{\text{jac}},\ \ell_u^{\text{hist}},\ \ell_u^{\text{future}}\right]^\top$$

Standardization: $x'_{u,j} = (x_{u,j} - \mu_j)/\sigma_j$ (z-score, not Gaussian assumption).

Clustering: KMeans on standardized features.

Each cluster $z \in \{0, \ldots, K-1\}$ gets its own bottleneck adapter $(W_{1,z}, W_{2,z}, b_{1,z}, b_{2,z})$.

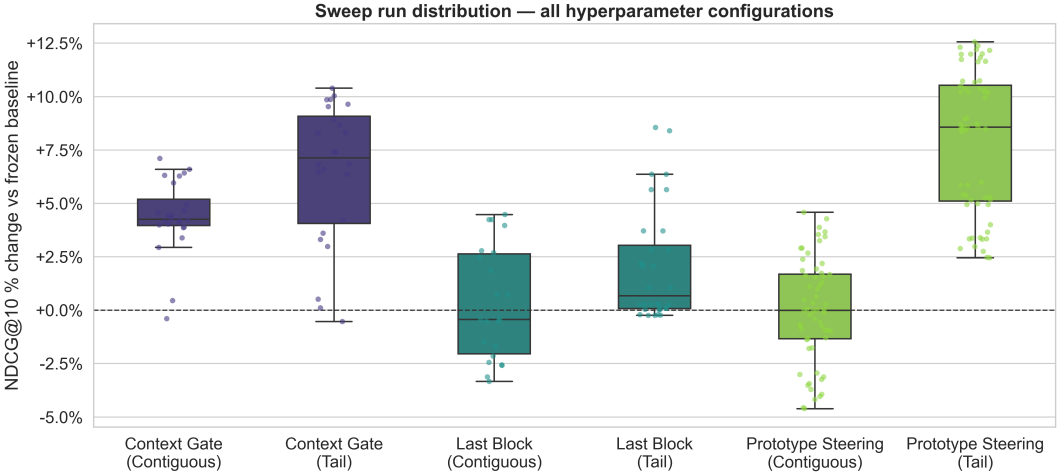
Why this matters

Different users may drift in different directions (e.g., one user shifts from pop to jazz, another from jazz to classical). Cluster-specific adapters can specialize their residual correction per group rather than applying one-size-fits-all.

Limitation

Works well for a *cohesive* high-drift group. Fails on heterogeneous general populations — conflicting drift signals overwhelm small per-cluster adapters.

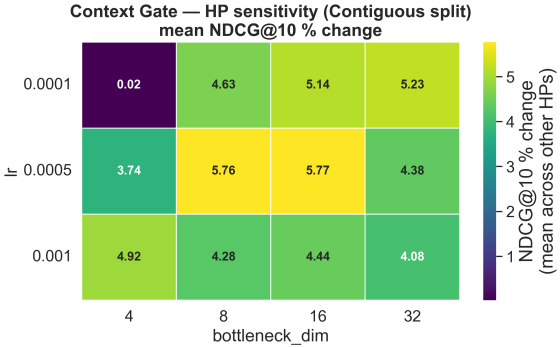
Appendix: Sweep Run Distribution



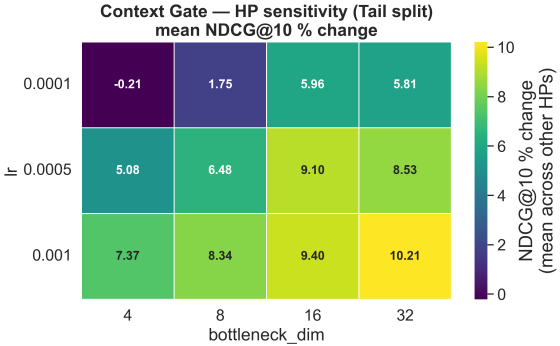
Distribution of NDCG@10 across *all* configurations in each sweep. Prototype Steering (Contiguous) is the only case with a substantial fraction of runs below zero — driven entirely by $K \geq 5$ configurations.

Appendix: HP Sensitivity Heatmaps — Context Gate

Contiguous



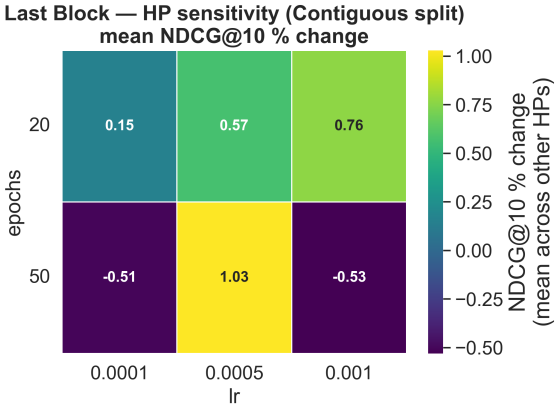
Tail



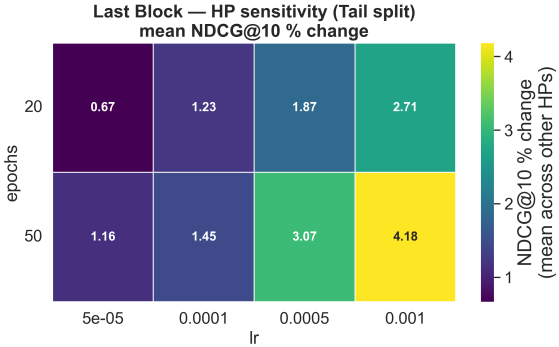
Mean NDCG@10 (%) averaged over epoch settings. lr=1e-4 nearly eliminates gain on contiguous. Tail optimum shifts to lr=1e-3, bd=32.

Appendix: HP Sensitivity Heatmaps — Last Block FT

Contiguous



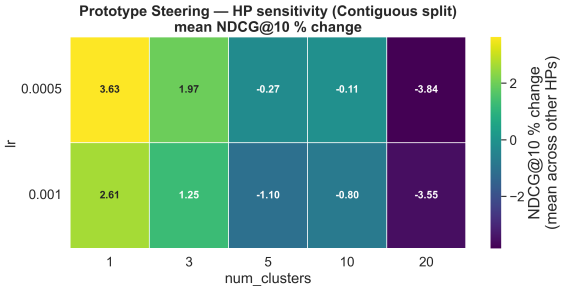
Tail



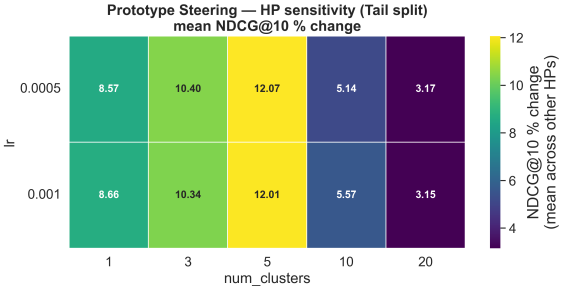
ep=50 with lr=5e-4 is the only safe combination on contiguous. lr=1e-3 is harmful at 50 epochs on contiguous but the best option on tail.

Appendix: HP Sensitivity Heatmaps — Prototype Steering

Contiguous



Tail



Cluster count K dominates; lr has comparatively little effect once K is fixed. Peak: $K = 1$ (contiguous), $K = 5$ (tail).